

Implementing `pick_place_node.cpp`

A ROS 2 + MoveIt 2 + C++ Lecture for Robot Manipulator Software Development

Target: UR16e · MoveIt 2 · ros2_control · Gazebo

Table of Contents

- | | |
|---|--|
| 1. Purpose of the Pick-and-Place Node | 11. IK computation and seed propagation |
| 2. Role of <code>pick_place_node</code> within the ROS 2 system | 12. Selecting collision-free IK solutions |
| 3. Manipulator control flow with MoveIt 2 | 13. Path planning and trajectory generation |
| 4. Overview of the full code structure | 14. Gripper open/close control |
| 5. Node initialization and parameter design | 15. Trajectory execution and failure handling |
| 6. <code>MoveGroupInterface</code> initialization | 16. <code>experiment_mode</code> and planner branching structure |
| 7. <code>PlanningSceneMonitor</code> and collision checking | 17. Integration point with a trajectory optimization server |
| 8. Action-server-based pick/place command handling | 18. Unit testing and simulation validation strategy |
| 9. Reading the current robot state | 19. Common issues and debugging methods |
| 10. Creating target poses and pre-grasp/pre-place poses | 20. Summary — full development roadmap |

1. Purpose of the Pick-and-Place Node

The node exposes two ROS 2 **action servers** — `/pick` and `/place` — that take a 6-DOF target pose (in `base_link`) of the object or destination and execute a full grasp/release cycle:

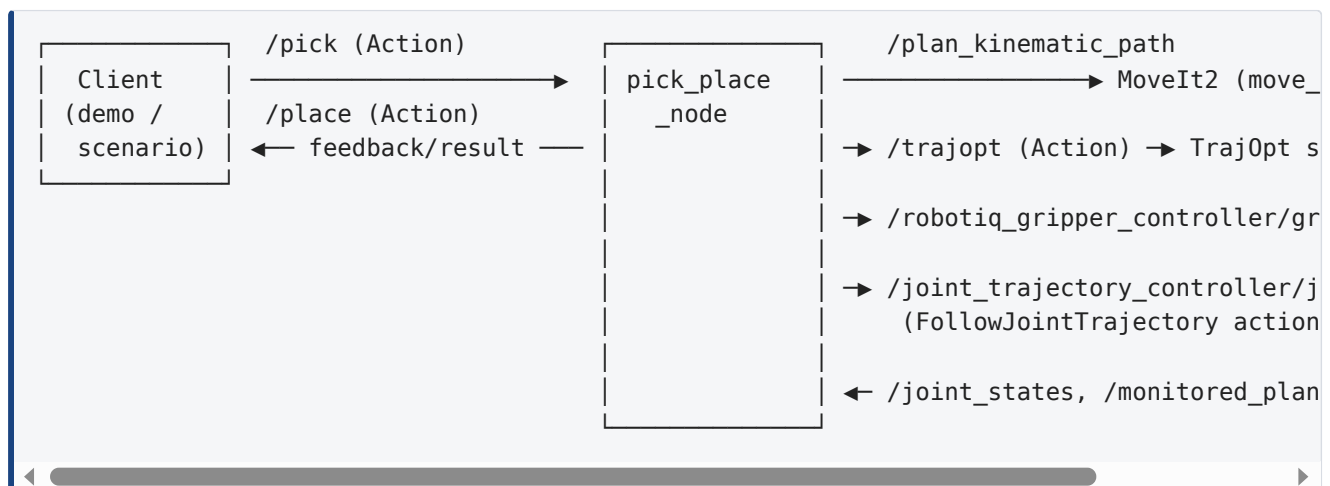
```
pre_grasp → grasp → lift → pre_place → place → retreat → home
```

Design philosophy:

1. **Decouple geometry from motion.** All IK is solved *before* any motion command. If IK is infeasible we abort early — no half-executed motions.
2. **Same IK branch from pre-grasp to grasp.** Use the grasp IK solution as the seed for the pre-grasp IK so the robot doesn't flip an elbow between approach and grasp.
3. **Three planning back-ends behind one interface:** `rrt_only`, `trajopt_only`, `rrt_trajopt`. Same outer API, selected at runtime via the `experiment_mode` parameter.
4. **Collision-aware approach.** Self/world collision is validated for the candidate IK *and* a discretized descent path before motion begins.

Slide summary: The pick-place node is a high-level skill: it converts a single pose goal into a validated, collision-free, optimization-aware joint-space trajectory.

2. Role of `pick_place_node` Within the ROS 2 System



- **Upstream:** the pick action consumer is typically a scenario script or demo controller.
- **Downstream:** `ros2_control` (`FollowJointTrajectory`), `Robotiq` gripper action, optional `TrajOpt` server.
- **Lateral:** a `PlanningSceneMonitor` subscribes to `/monitored_planning_scene` to see the same world that `move_group` sees.

Slide summary: The node is an orchestrator: it consumes a pose Action, fan-outs to `MoveIt2`, `TrajOpt` and the gripper, and reports back through `feedback/result`.

3. Manipulator Control Flow with MoveIt 2

```
goal pose (base_link)
├─▶ applyDownwardOrientation()      ; force tool Z = -world Z, preserve current yaw
├─▶ computeCollisionFreeIK()        ; goal IK + self/world collision check
    (random seed restarts if anchor IK lands on a colliding branch)
├─▶ findCollisionFreePreApproachIK() ; pre-pose IK seeded with anchor solution
    + validateJointSpaceDescent()  : joint-space interpolation collision check
├─▶ planAndExecuteToJoints()        ; experiment_mode branch
    │
    │ ┌─ rrt_only      → MoveGroupInterface::plan + execute
    │ ┌─ rrt_trajopt   → plan, then send plan as initial guess to TrajOpt server
    │ │               → execute optimized trajectory (FollowJointTrajectory)
    │ └─ trajopt_only  → no RRT; q_start/q_end → TrajOpt server (Hermite init)
├─▶ approach (verticalCartesianMove | cartesianMove | joint plan)
    grasp/release
    retreat (best-effort)
```

Slide summary: MoveIt 2 is used not as a black box but as four explicit services: state query, IK, planning-scene collision, and plan-execute.

4. Overview of the Full Code Structure

Layer	Responsibility	Key entity
Helpers (file-scope)	L2 norm, joint-order remapping, trajectory metrics, CSV logger	<code>computeL2Norm</code> , <code>reorderToUrOrder</code> , <code>TrajectoryMetrics</code> , <code>CsvLogger</code>
PickPlaceNode (ctor)	Parameters, MoveGroup, PSM, callback groups, action servers, publishers, worker thread	constructor
Geometry layer	Downward orientation, pose construction	<code>applyDownwardOrientation</code>
IK layer	Pose→joint with seed propagation, collision filtering	<code>computeIKForPose</code> , <code>computeCollisionFreeIK</code> , <code>findCollisionFreePreApproachIK</code>
Collision layer	PSM lock + collision query, descent validation	<code>inCollision</code> , <code>validateJointSpaceDescent</code>
Motion layer	Plan/execute branching, Cartesian moves	<code>planAndExecuteToJoints</code> , <code>cartesianMove</code> , <code>verticalCartesianMove</code>
Optimizer layer	TrajOpt action client (with and without RRT)	<code>runWithTrajopt</code> , <code>runTrajoptOnly</code>
Skill layer	Full Pick / Place sequences	<code>executePick</code> , <code>executePlace</code>
Concurrency layer	Worker queue, abort flag, cancel service	<code>workerLoop</code> , <code>enqueueTask</code> , <code>abort_requested_</code>
<code>main</code>	Two-node bootstrap (MoveGroup node + PickPlace node, separate executors)	<code>main</code>

Slide summary: The file is layered: helpers → IK/collision → motion/optimizer → skill. Each layer only depends on the layers below.

5. Node Initialization and Parameter Design

Learning objective

Understand why this node uses `allow_undeclared_parameters(true)` , what each parameter group controls, and how parameters are read **lazily at every call** (not cached) so an external benchmark runner can switch planners between iterations.

Required theory

- `rclcpp::NodeOptions().allow_undeclared_parameters(true)` — avoids a race condition where `set_parameters` arrives before the constructor finishes declaring.
- Parameter groups: kinematics, planner config, gripper, TrajOpt, experiment mode, approach strategy, descent validation.

Skeleton

```
class PickPlaceNode : public rclcpp::Node {
public:
    explicit PickPlaceNode(std::shared_ptr<rclcpp::Node> mg_node)
        : Node("pick_place_node",
              rclcpp::NodeOptions().allow_undeclared_parameters(true))
    {
        declare_parameter<std::string>("arm_group", "ur_manipulator");
        declare_parameter<double>("pre_grasp_offset", 0.10);
        declare_parameter<std::string>("planner_id", "RRTConnectkConfigDefault");
        declare_parameter<bool>("use_trajopt", false);
        declare_parameter<std::string>("experiment_mode", "rrt_trajopt");
        declare_parameter<std::string>("approach_strategy", "vertical_cartesian");
        // ... ~30 more in pick_place_params.yaml
    }
};
```

Verification

`ros2 param list /pick_place_node` and `ros2 param set /pick_place_node planner_id RRTstarkConfigDefault`. Next plan iteration should report the new planner in CSV.

Common errors

- Forgetting `allow_undeclared_parameters` → runner crashes at startup.
- Reading parameters once into members → runtime planner switching breaks.

Slide summary: *Declare permissively; read lazily; switch planners by parameter, not by recompile.*

6. MoveGroupInterface Initialization

Learning objective

Initialize `MoveGroupInterface` **on a separate node** with an independent `SingleThreadedExecutor`.

Why two nodes?

`MoveGroupInterface` makes synchronous service calls. If you spin it on the same executor as your action callbacks, you'll deadlock — the callback waits for a service that needs the same executor to spin.

Skeleton — `main`

```
int main(int argc, char ** argv) {
    rclcpp::init(argc, argv);
    auto mg_node = rclcpp::Node::make_shared("pick_place_move_group_interface");

    rclcpp::executors::SingleThreadedExecutor mg_exec;
    mg_exec.add_node(mg_node);
    std::thread mg_thread([&]{ mg_exec.spin(); });

    auto node = std::make_shared<PickPlaceNode>(mg_node);
    rclcpp::executors::MultiThreadedExecutor exec;
    exec.add_node(node);
    exec.spin();
}
```

Skeleton — inside ctor

```
move_group_ = std::make_shared<MoveGroupIface>(mg_node, arm_group);
move_group_ ->setPlannerId(planner_id);
move_group_ ->setMaxVelocityScalingFactor(0.3);
move_group_ ->setMaxAccelerationScalingFactor(0.3);
move_group_ ->setPoseReferenceFrame("base_link");
move_group_ ->setWorkspace(-2, -2, -2, 2, 2, 2);
```

Verification

`RCLCPP_INFO("Planning frame: %s", move_group_ ->getPlanningFrame().c_str());` should print `world` (or `base_link`, depending on SRDF).

Common errors

- Single-node design → deadlock on first `getCurrentState()`.
- Forgetting `setPoseReferenceFrame` → pose goals get reinterpreted in the planning frame.

Slide summary: *MoveGroupInterface needs its own spinning executor. Put it on a separate node thread.*

7. PlanningSceneMonitor and Collision Checking

Learning objective

Reuse the **same** robot_description and planning scene that move_group uses, so collision predictions match the planner's.

Required theory

- `PlanningSceneMonitor` (PSM) listens to `/monitored_planning_scene` and `/joint_states`.
- `LockedPlanningSceneRO ps(psm_)` gives a read-only snapshot. **Never hold the lock across motion calls.**
- `ps->checkCollision(req, res, state)` runs FCL with the SRDF self-collision matrix.

Skeleton

```
psm_ = std::make_shared<planning_scene_monitor::PlanningSceneMonitor>(
    mg_node, "robot_description");
psm_->startSceneMonitor("/monitored_planning_scene");
psm_->startWorldGeometryMonitor();
psm_->startStateMonitor("/joint_states");
psm_->requestPlanningSceneState("/get_planning_scene");

bool inCollision(const std::vector<double>& q, const std::string& group) {
    if (!psm_) return false;
    planning_scene_monitor::LockedPlanningSceneRO ps(psm_);
    auto state = ps->getCurrentState();
    state.setJointGroupPositions(state.getJointModelGroup(group), q);
    state.update();
    collision_detection::CollisionRequest req; req.group_name = group;
    collision_detection::CollisionResult res;
    ps->checkCollision(req, res, state);
    return res.collision;
}
```

Verification

Move an obstacle into the robot's path in Gazebo; `inCollision()` should flip to `true`.

Common errors

- Creating a *separate* `RobotModel` → SRDF self-collision matrix not loaded → false negatives.
- Calling `move_group_->execute()` while holding the lock → deadlock.

Slide summary: PSM gives you a fast, planner-consistent "would this joint state collide?" query. Use it to filter IK candidates before planning.

8. Action-Server-Based Pick/Place Command Handling

Learning objective

Build a `rcpp_action::Server` that **enqueues** instead of detaching, supports cancel, and handles goal-state transitions correctly.

Required theory

Three callbacks: `goal`, `cancel`, `accepted`. The trap to avoid: in `accepted`, the naive pattern is `std::thread(...).detach()`, which makes safe shutdown impossible (use-after-free on the node).

Skeleton

```
pick_server_ = rclcpp_action::create_server<Pick>(
    this, "pick",
    [this](const auto&, auto) { return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE; },
    [this](auto gh) {
        abort_requested_.store(true);
        move_group_->stop();
        return rclcpp_action::CancelResponse::ACCEPT;
    },
    [this](auto gh) {
        enqueueTask([this, gh]{ executePick(gh); }); // not detach()
    },
    rcl_action_server_get_default_options(), pick_cbg_);
```

Worker queue

```
void workerLoop() {
    while (true) {
        std::function<void()> task;
        {
            std::unique_lock<std::mutex> lk(queue_mutex_);
            queue_cv_.wait(lk, [&]{ return shutting_down_ || !task_queue_.empty(); });
            if (shutting_down_ && task_queue_.empty()) return;
            task = std::move(task_queue_.front()); task_queue_.pop();
        }
        try { task(); } catch (...) { /* log */ }
    }
}
```

Goal-state-aware terminal transitions

```
auto abort_with = [&](const std::string& msg) {
    result->success = false; result->message = msg;
    if (gh->is_canceling()) gh->canceled(result);
    else gh->abort(result);
};
```

Why: Calling `abort()` on a `CANCELING` goal raises `IllegalTransitionException` → SIGABRT.

Common errors

- `std::thread(...).detach()` — looks fine in tests, crashes on shutdown.
- Treating cancel as a one-time signal — you need to check **before every long operation**.

***Slide summary:** Action servers should enqueue, not detach. Always pick the right terminal transition based on `is_canceling()`.*

9. Reading the Current Robot State

Learning objective

Use `move_group_->getCurrentState(timeout)` correctly and convert into the right joint order.

Required theory

- `RobotState::copyJointGroupPositions(jmg, vec)` returns joints **in** `jmg->getVariableNames()` **order**, which may not match the hardware controller order.
- For TrajOpt, you must remap to the UR controller order.

Skeleton

```
auto rs = move_group_->getCurrentState(2.0);
if (!rs) { /* fail */ }
const auto* jmg = rs->getJointModelGroup(arm_group_name);
std::vector<double> q_moveit;
rs->copyJointGroupPositions(jmg, q_moveit);
auto q_ur = reorderToUrOrder(q_moveit, jmg); // controller order
```

Verification

`echo /joint_states` and compare to the printed `q_ur`. They must match position-by-position.

Common errors

- Assuming joint order = SRDF order. **It is not.**
- Calling `getCurrentState` without a timeout under a heavy executor — returns null silently.

***Slide summary:** Joint vectors carry an implicit order. Always remap to hardware order before talking to controllers or TrajOpt.*

10. Creating Target Poses and Pre-Grasp/Pre-Place Poses

Learning objective

Build a grasp pose that (a) preserves current EE yaw and (b) has tool-Z pointing down, then derive a pre-grasp pose by simple Z-offset.

Required theory — `applyDownwardOrientation`

1. Get current `base_R_ee`.
2. Project the current EE X axis onto the horizontal plane.
3. Build a rotation matrix `[x, y, -z_world]` and convert to a quaternion.
4. Fallback to fixed quaternion parameters if state read fails.

Skeleton

```
geometry_msgs::msg::Pose grasp_pose = applyDownwardOrientation(goal->pick_pose);
geometry_msgs::msg::Pose pre_grasp_pose = grasp_pose;
pre_grasp_pose.position.z += pre_grasp_offset; // e.g. 0.10 m
```

Common errors

- Hard-coding orientation → robot rotates wrist unnecessarily on every grasp.
- Setting pre-grasp by Cartesian offset along EE-Z instead of world-Z → fails when the EE is tilted. (Uncertain: in this codebase the offset is along world Z; if you instead want along tool Z, transform the offset by `world_R_ee`.)

Slide summary: Pre-grasp = world-Z lift of the grasp pose. Grasp orientation = current yaw, pinned downward.

11. IK Computation and Seed Propagation

Learning objective

Compute IK so that pre-grasp and grasp end up on the **same IK branch** (no elbow flip mid-approach).

Required theory

- `RobotState::setFromIK(jmg, pose, timeout, validity_cb, opts, cost_fn)` solves IK with an optional cost function (MoveIt 2 ≥ Humble; uncertain for older distros).
- Two mechanisms keep the branch:
 1. **Seed state** — the search starts from a known joint configuration.
 2. **L2 cost** — $\text{cost} = w \cdot \sum (q_i - \text{seed}_i)^2$ so candidates close to the seed win.
- IK can be retried with random restarts on failure.

Skeleton — `computeIKForPose`

```
auto cost_fn = [w_l2, seed_state](
    const auto&, const moveit::core::RobotState& s,
    const moveit::core::JointModelGroup* jmg, const auto&) {
    std::vector<double> p;
    s.copyJointGroupPositions(jmg, p);
    return w_l2 * computeL2Norm(p, seed_state);
};

for (int att = 0; att < max_attempts && !found; ++att) {
    auto st = std::make_shared<moveit::core::RobotState>(*rs);
    if (att == 0) st->setJointGroupPositions(jmg, seed_state);
    else         st->setToRandomPositions(jmg);
    found = st->setFromIK(jmg, target_world, ik_timeout, {}, opts, cost_fn);
}
```

Seed propagation pattern (key design)

```
// (1) grasp IK — seed = current joints
computeIKForPose(grasp_pose, current_state, grasp_solution, ...);
// (2) pre-grasp IK — seed = grasp solution
computeIKForPose(pre_grasp_pose, grasp_solution, pre_grasp_solution, ...);
```

Verification

Log `L2` vs `seed`. If `L2 >  $\pi$` , you're on a different IK branch — bad.

Common errors

- Solving pre-grasp first, then grasp → no branch consistency.
- Using `setFromIK` without converting the pose to the planner's world frame (the code does `world_T_base * target_in_base`).

Slide summary: *IK without a seed is just a sample. With seed + L2 cost, it becomes a deterministic branch selector.*

12. Selecting Collision-Free IK Solutions

Learning objective

Distinguish IK feasibility from IK **safety**. A kinematically valid IK can still be in self-collision.

Two functions in the file

- `computeCollisionFreeIK` — anchor (grasp/place) pose; rejects colliding solutions, retries with random restarts.
- `findCollisionFreePreApproachIK` — pre-pose; in addition, validates the straight-line **joint-space descent** from candidate → anchor is collision-free across N discrete steps.

Skeleton — descent validation

```
bool validateJointSpaceDescent(const std::vector<double>& a,
                              const std::vector<double>& b, int N) {
    for (int k=0; k<=N; ++k) {
        double alpha = double(k)/N;
        std::vector<double> q(a.size());
        for (size_t j=0; j<a.size(); ++j) q[j] = (1-alpha)*a[j] + alpha*b[j];
        if (inCollision(q, group)) return false;
    }
    return true;
}
```

Important design note

Defensively pre-check the **anchor** itself for collision. If the anchor is colliding, every pre-approach candidate will fail at `alpha=1.0` — infinite retries for nothing.

Common errors

- Treating IK success = safe. Always run `inCollision` afterwards.
- Picking `descent_check_steps` too small (≤ 2) → missing intermediate collisions.

Slide summary: Always check the anchor first; then check the path. IK feasibility is necessary but not sufficient.

13. Path Planning and Trajectory Generation

Learning objective

Use `MoveGroupInterface::plan()` with a **joint target** (not pose target) once IK is locked in.

Required theory

- `setJointValueTarget(joint_vec)` + `plan()` runs OMPL's `RRTConnect` (or whatever `planner_id` you set) and time-parameterizes the result with IPTP/TOTG.
- Result is a `moveit_msgs::RobotTrajectory` ready for `execute()`.
- `setPlannerId()` is called **per plan** so a parameter change takes effect within one cycle.

Skeleton — `planAndExecuteToJoints` (`rrt_only` branch)

```
move_group->setPlannerId(get_parameter("planner_id").as_string());
move_group->setJointValueTarget(joint_target);

MoveGroupIface::Plan plan;
auto t0 = steady_clock::now();
if (!move_group->plan(plan)) { /* fail */ }
rec.rrt_planning_sec = durationSec(t0, steady_clock::now());
auto er = move_group->execute(plan);
```

Cartesian variants

- `computeCartesianPath(waypoints, eef_step, jump_threshold, traj)` returns a *fraction* (0–1) describing how much was achieved. Reject below `min_fraction` (typical 0.95).
- `verticalCartesianMove` builds N waypoints with `x,y,orientation` fixed to end pose and `z` linearly interpolated — guarantees a straight vertical descent that joint planning would not.

Common errors

- Calling `setPoseTarget` instead of `setJointValueTarget` after you computed IK. Wastes a second IK and may pick a different branch.
- Using `computeCartesianPath` with `jump_threshold > 0` and tight `eef_step` → spurious failures at IK boundaries.

Slide summary: Once IK is committed, switch to joint targets. Use Cartesian only for short straight segments where you must guarantee EE geometry.

14. Gripper Open/Close Control

Learning objective

Drive a gripper through `control_msgs/action/GripperCommand` with a future-based blocking wrapper.

Required theory

- The Robotiq controller exposes `/<name>/gripper_cmd` of type `control_msgs/action/GripperCommand`.
- The gripper callback group must be **Reentrant**: pick/place sequences block on the arm while the gripper runs.

Skeleton

```
bool controlGripper(double pos, double max_effort, double timeout) {
    auto promise = std::make_shared<std::promise<bool>>();
    auto future = promise->get_future();

    GripperCommand::Goal goal{};
    goal.command.position = pos; goal.command.max_effort = max_effort;

    rclcpp_action::Client<GripperCommand>::SendGoalOptions opts;
    opts.result_callback = [promise](const auto& res) {
        promise->set_value(res.code == rclcpp_action::ResultCode::SUCCEEDED);
    };
    gripper_client_->async_send_goal(goal, opts);
    return future.wait_for(seconds(timeout)) == future_status::ready && future.get();
}
```

Common errors

- `MutuallyExclusive` callback group on the gripper → deadlock with the pick action.
- Not handling `goal_response_callback` rejection → `promise` never set → eternal wait.

Slide summary: Wrap async actions with promise/future, and put cross-cutting callbacks on a Reentrant group.

15. Trajectory Execution and Failure Handling

Learning objective

Execute trajectories in a **closed loop** and translate every failure mode into a clear, attributable label.

Required theory — closed vs open loop

- `move_group_->execute(traj)` uses `FollowJointTrajectory` internally; it blocks until the controller reports completion.
- The original code published to `/joint_trajectory_controller/joint_trajectory` and `sleep_for(T_opt + margin)`. That was *open loop* — hardware lag silently violated assumptions. The Stage 1.1 fix wraps the `TrajOpt` result in `RobotTrajectory` and uses `execute()` instead.

Failure label discipline

The Pick sequence emits six distinct failure labels:

- `grasp IK failed`
- `pre_grasp IK failed`

- `pre_grasp planning failed`
- `pre_grasp execution failed`
- `approach planning failed`
- `approach execution failed`

```
if (!pre_ok) {
    bool was_planning = pre_rec.message.find("planning") != std::string::npos;
    abort_with(was_planning ? "pre_grasp planning failed: " + pre_rec.message
                           : "pre_grasp execution failed: " + pre_rec.message);
    return;
}
```

Best-effort phase

The **retreat** is best-effort: log a warning, do not abort — the object has already been grasped/released.

Common errors

- Returning the same "failed" message for every failure mode → impossible to triage from CSV.
- Aborting on retreat failure → robot stops mid-air still holding the object.

Slide summary: Label every failure by stage and by sub-step. Make retreat best-effort.

16. `experiment_mode` and Planner Branching Structure

Learning objective

Build one outer function (`planAndExecuteToJoints`) that dispatches to three planning back-ends so you can A/B/C compare them per-step.

Mode matrix

<code>experiment_mode</code>	Pipeline	Fallback
<code>rrt_only</code>	<code>setJointValueTarget</code> → <code>plan</code> (RRTConnect) → <code>execute</code>	none
<code>trajopt_only</code>	IK only → <code>q_start/q_end</code> (UR order) → TrajOpt server → <code>execute</code>	none
<code>rrt_trajopt</code>	RRT plan → TrajOpt with RRT as initial guess → <code>execute</code>	RRT plan on TrajOpt failure

Skeleton (dispatch)

```
const std::string mode = get_parameter("experiment_mode").as_string();
move_group_>setPlannerId(get_parameter("planner_id").as_string());

if (mode == "trajopt_only") {
    auto q0 = reorderToUrOrder(current, jmg);
    auto q1 = reorderToUrOrder(joint_target, jmg);
    return runTrajoptOnly(q0, q1, step_name, rec);
}

MoveGroupIface::Plan plan;
move_group_>setJointValueTarget(joint_target);
if (!move_group_>plan(plan)) return false;

if (mode == "rrt_only") return bool(move_group_>execute(plan));

// rrt_trajopt
bool ok = runWithTrajopt(plan, true, current, joint_target, step_name, rec);
if (!ok) { rec.fallback_used = true; ok = bool(move_group_>execute(plan)); }
return ok;
```

Verification

The CSV columns `planner_id` and `experiment_mode` should reflect what was actually used at run time, with `fallback_used=true` rows visible when TrajOpt fails.

Common errors

- Switching mode but forgetting `setPlannerId` — old planner persists.
- Sending `q_start/q_end` in MoveIt variable order to a TrajOpt server expecting UR order.

Slide summary: Mode is one parameter, one dispatch, three back-ends — perfect for fair comparison.

17. Integration Point with a Trajectory Optimization Server

Learning objective

Extract an RRT plan's joint waypoints as an initial guess for a non-linear trajectory optimizer (`/trajopt` action).

Required theory

The action interface `pick_place_module/action/TrajOpt` (uncertain — verify the exact `.action` schema in the repo) carries:

- `q_start`, `q_end` (UR order)

- `num_waypoints`, `q_waypoints[]`, `timestamps[]` — flattened ($K \times 6$)
- `t_init`, `cheb_degree`, `use_reduced`, `use_free_t`
- Returns: `optimized_trajectory` (`JointTrajectory`), `t_opt`, `cost`, torque metrics, etc.

Skeleton — extracting RRT waypoints

```
TrajOpt::Goal g;
g.t_init = t_init; g.cheb_degree = N_cheb;
g.use_reduced = use_reduced; g.use_free_t = use_free_t;
g.num_waypoints = jt.points.size();

for (auto& pt : jt.points)
    g.timestamps.push_back(pt.time_from_start.sec + pt.time_from_start.nanosec*1e-9);

for (auto& pt : jt.points)
    for (int j=0; j<6; ++j) {
        int k = joint_idx[j];          // map UR→plan order
        g.q_waypoints.push_back(pt.positions[k]);
    }

g.q_start = q_start_ur; g.q_end = q_end_ur;
trajopt_client->async_send_goal(g, opts);
```

Closed-loop execution of the optimized trajectory

```
moveit_msgs::msg::RobotTrajectory rt;
rt.joint_trajectory = result->optimized_trajectory;
auto er = move_group_->execute(rt);
```

Where students often go wrong

- Sending waypoints in MoveIt's variable order to TrajOpt's UR order schema → silent joint shuffling.
- Hermite-only mode (`num_waypoints=0`) is fine in free space, but in cluttered scenes you need the RRT initial guess.
- Treating "NLP did not fully converge" (`res->success == false`) as failure. The file's policy is *warn + continue* using the partial solution. (Uncertain — verify this is what your TrajOpt server returns in such cases.)

Slide summary: RRT gives feasibility; TrajOpt smooths it. Bridge them by flattening the plan into the action's `q_waypoints` array.

18. Unit Testing and Simulation Validation Strategy

Stage A — Static checks (no robot)

- **Parameter file load:** `ros2 param dump` after launch must match `pick_place_params.yaml`.
- **IK round-trip test:** call `applyDownwardOrientation` + `computeIKForPose` on a known reachable pose; assert the resulting joint values produce that pose via FK (within 1 mm / 1°).

Stage B — RViz only

- Set `experiment_mode=rrt_only` and `approach_strategy=joint`.
- Send a `/pick` goal with a pose 30 cm in front of the base. Watch `/ee_path/planned` (Path topic) and confirm the trajectory in RViz.

Stage C — Gazebo + MoveIt 2 + ros2_control

- Bring up Gazebo with the UR16e + Robotiq gripper.
- Send pick/place via the demo scripts.
- Validate:
 1. Gripper opens before approach.
 2. Vertical descent is straight in the RViz EE trail.
 3. Object follows the EE (Gazebo attach plugin or `moveit_msgs::AttachedCollisionObject`).

Stage D — Benchmark loop

- Run a runner that toggles `planner_id` and `experiment_mode` between trials.
- Inspect `~/.ros/pick_place_exp/<timestamp>_results.csv` and verify `planner_id`, `experiment_mode`, and `success` columns reflect the runner's intent.

Common errors

- Validating IK only kinematically — collisions only show up in PSM, which only works after `startSceneMonitor`.
- Trusting OMPL success in RViz when `ros2_control` isn't loaded — `execute` will fail silently.

Slide summary: *Static → RViz → Gazebo → benchmark. Each stage catches a different class of bug.*

19. Common Issues and Debugging Methods

Symptom	Likely cause	Where to look
<code>getCurrentState</code> failed	Joint state not arriving fast enough	Check <code>/joint_states</code> rate; raise the timeout arg
IK FAILED on a clearly reachable pose	Pose given in wrong frame	<code>applyDownwardOrientation</code> + <code>world_T_base</code> * <code>target_in_base</code>
IK ok but descent COLLISION at step 0/N (alpha=0)	Pre-pose IK itself is colliding	<code>findCollisionFreePreApproachIK</code> retries with random seeds; raise <code>descent_max_ik_retries</code>
descent COLLISION at step N/N (alpha=1.0)	Anchor IK itself is colliding	Raise <code>descent_max_ik_retries</code> for the anchor or fix the target pose
Cartesian path 30 %	<code>eef_step</code> too coarse or pose just out of reach	Lower <code>cartesian_eef_step</code> to 0.005; if still failing, the pose is unreachable from pre-pose along a straight line — use <code>joint</code> strategy
TrajOpt server not available	<code>/trajopt</code> not launched	<code>ros2 action list grep trajopt</code> ; raise <code>trajopt_server_timeout_sec</code>
Node crashes on shutdown (SIGABRT)	Goal state vs terminal-transition mismatch	Verify both <code>executePick</code> and <code>executePlace</code> use the <code>is_canceling()</code> switch
Elbow flips between pre-grasp and grasp	Seed not propagated	Confirm pre-grasp IK seed = <code>grasp_solution</code> , not <code>current_state</code>

Slide summary: Most production failures are IK/collision misclassifications, not planner failures. Read the failure label first.

20. Summary — Full Development Roadmap

- Skeleton node** — declare parameters, two-node `main`, empty action servers.
- MoveGroupInterface** — initialize, set planner/scaling/frame/workspace.
- PlanningSceneMonitor** — start three monitors; implement `inCollision()`.
- Gripper client** — Reentrant CBG + future-based `controlGripper`.
- Action servers with worker queue** — `enqueueTask`, `workerLoop`, abort flag, cancel service.
- `applyDownwardOrientation` — preserve EE yaw, force tool-Z down.
- `computeIKForPose` — IK with seed + L2 cost (base→world transform inside).
- Collision-aware IK** — `computeCollisionFreeIK`, `validateJointSpaceDescent`, `findCollisionFreePreApproachIK`.
- `planAndExecuteToJoints` — single dispatch for `rrt_only` / `rrt_trajopt` / `trajopt_only`.
- Cartesian helpers** — `cartesianMove`, `verticalCartesianMove`.
- TrajOpt clients** — `runWithTrajopt` (RRT seed) and `runTrajoptOnly` (Hermite).

12. `executePick / executePlace` — seven-step sequence with cancel checkpoints and labeled failures.
 13. **CSV + EE path publishers** — observability for benchmarking.
 14. **Gazebo + benchmark validation** — sweep planners and modes; inspect CSV.
-

End of Lecture Notes — `pick_place_node.cpp`